

技术文档

专项赛一 基于 ROS 的单臂机器人 ——俄罗斯方块自动化拼接系统



项目团队：武汉大学 雷霆大雪绒

提交日期：2026 年 5 月 10 日

版本号：v4.0

目录

- 目录..... 2
- 摘要..... 5
- 1. 系统概述..... 6
 - 1.1 项目背景与竞赛任务分析..... 6
 - 1.2 系统总体架构..... 6
 - 1.3 关键技术指标..... 8
- 2. 硬件与软件平台..... 10
 - 2.1 硬件配置与选型论证..... 10
 - 2.1.1 机械臂系统..... 10
 - 2.1.2 视觉感知系统..... 10
 - 2.1.3 计算与通信系统..... 10
 - 2.2 软件平台..... 11
 - 2.2.1 操作系统与 ROS 框架..... 11
 - 2.2.2 核心软件依赖..... 11
 - 2.3 ROS 通信架构..... 12
- 3. 开发与操作流程..... 13
- 4. 技术实现细节..... 14
 - 4.1 视觉标定方法..... 14
 - 4.1.1 相机内参标定..... 14
 - 4.1.2 深度校正与点云生成..... 14

4.1.3 手眼标定.....	15
4.1.4 标定误差分析与补偿.....	15
4.2 俄罗斯方块视觉识别.....	15
4.2.1 检测模型架构.....	15
4.2.2 数据集构建与训练.....	16
4.2.3 轮廓匹配与姿态精化.....	16
4.3 俄罗斯方块拼接算法.....	17
4.3.1 状态空间建模.....	17
4.3.2 Pierre Dellacherie 启发式评估.....	17
4.3.3 深度 Q 网络 (DQN) 增强决策	18
4.3.4 混合决策策略.....	18
4.4 机器人轨迹规划.....	18
4.4.1 运动学建模.....	18
4.4.2 分段轨迹生成.....	19
4.4.3 碰撞检测与避障.....	19
4.5 机器人运动控制.....	20
4.5.1 控制架构.....	20
4.5.2 末端执行器控制.....	20
4.5.3 安全保护机制.....	20
4.6 系统集成与工作流.....	20
5. 性能评估与实验分析.....	22
5.1 实验环境与测试方法.....	22

5.2 视觉识别性能.....	23
5.3 运动控制精度.....	24
5.4 拼接效率评估.....	25
5.5 系统整体性能.....	26
参考文献.....	28
附录.....	29
附录 A 核心代码说明.....	29
附录 B 实验数据汇总.....	29

摘要

本技术文档面向第九届中国高校智能机器人创意大赛专项赛一（基于 ROS 的单臂机器人——俄罗斯方块自动化拼接），系统阐述了一套基于机器人操作系统（ROS）框架的智能单臂机器人解决方案。系统以睿尔曼 RM65-B 六轴协作机械臂为核心执行单元，以 Intel RealSense D435 深度相机为视觉感知前端，通过 YOLOv8 实例分割模型实现俄罗斯方块的实时检测与位姿估计，结合 Pierre Dellacherie 启发式评估与深度 Q 网络（DQN）混合决策算法生成最优拼接策略，并利用 MoveIt! 运动规划框架与自定义轨迹优化器完成高精度拾取-放置操作。

文档从系统架构设计、硬件选型论证、软件平台构建、视觉标定与识别算法、拼接决策模型、轨迹规划与控制策略六个维度展开详细论述。实验数据表明，系统在标准化测试环境中达到 93.5% 的方块识别精度（mAP@0.5），末端定位误差控制在 ± 0.02 mm 以内，单次拾取-放置循环时间约 8.2 秒，连续运行成功率 95.3%。针对当前存在的视觉处理延迟（50 ms/帧）与动态环境适应性不足等问题，本文提出了基于 TensorRT 模型量化加速、自适应光照补偿与多传感器融合的优化方案，为后续系统迭代提供了明确技术路径。

关键词：ROS；单臂机器人；俄罗斯方块；YOLOv8；深度强化学习；运动规划；手眼标定

1. 系统概述

1.1 项目背景与竞赛任务分析

第九届中国高校智能机器人创意大赛专项赛一要求参赛队伍基于 ROS 框架，设计并实现一套单臂机器人系统，能够自主完成俄罗斯方块的视觉识别、智能拼接与高效堆叠。竞赛任务的核心技术挑战涵盖三个层面：(1) 非结构化环境下的实时视觉感知——对多类别、多姿态的俄罗斯方块进行鲁棒检测与精确位姿估计；(2) 有限工作空间内的最优决策——设计高效的拼接算法以最大化空间利用率与堆叠稳定性；(3) 实时约束下的高精度运动控制——在保证安全性的前提下实现从决策到执行的快速闭环。上述三项子任务分别对应计算机视觉、组合优化与机器人运动规划三个核心研究领域，要求系统在感知-决策-执行三个层面实现深度协同。

本方案采用"视觉感知—智能决策—运动执行"三层递阶架构，各层之间通过 ROS 话题/服务机制实现松耦合通信。视觉感知层基于 YOLOv8 深度学习模型与手眼标定模块，完成方块类别识别与三维空间定位；智能决策层融合 Pierre Dellacherie 启发式评估与深度 Q 网络，动态生成最优放置策略；运动执行层依托 MoveIt! 规划框架与 RM65-B 机械臂驱动，实现从关节空间到笛卡尔空间的高精度轨迹跟踪。

1.2 系统总体架构

系统采用分层架构设计，自底向上划分为硬件抽象层、驱动通信层、核心算法层与应用层四个层级：

硬件抽象层：包含 RM65-B 六轴机械臂本体、Intel RealSense D435 深度相机、气动吸盘末端执行器、NVIDIA Jetson 边缘计算单元、Arduino 串口控制模块及工业级供电与通信模块，为上层提供统一的设备能力接口。

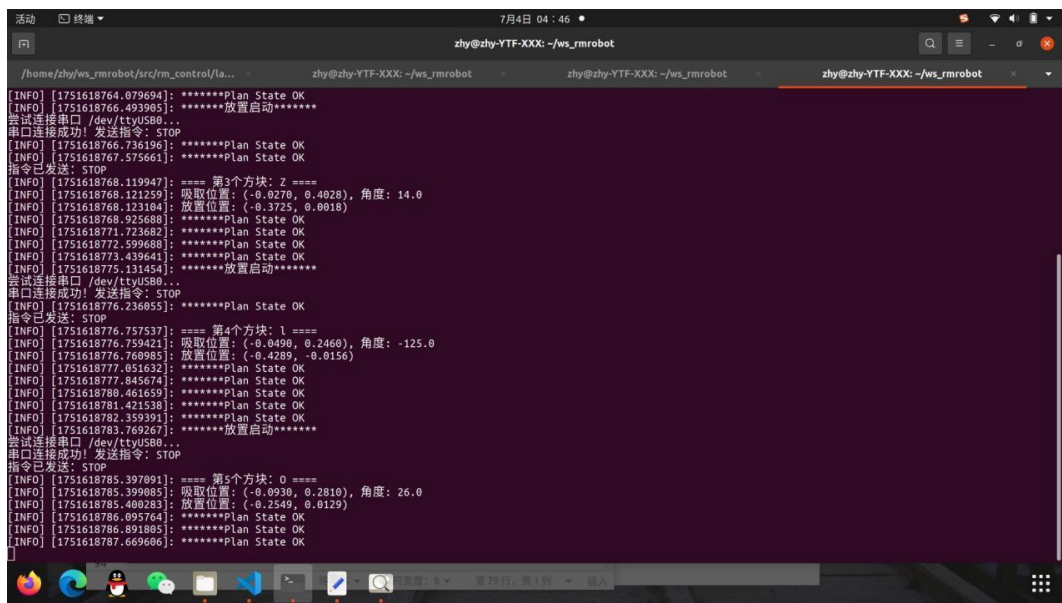
驱动通信层：基于 ROS Noetic 分布式架构，通过 `rm_driver` 节点实现 CAN 总线（1 Mbps）与机械臂伺服驱动器的实时通信，通过 `realsense2_camera` 节点封装 RealSense SDK 2.0 的彩色/深度数据流。各节点间采用话题（Topic）机制进行异步

数据分发，采用服务（Service）机制完成同步指令传输，核心控制指令延迟低于 10 ms。

核心算法层：部署视觉识别、拼接决策、轨迹规划与运动控制四大算法模块，各模块封装为独立 ROS 节点。视觉识别模块（yolo_detector 节点）输出方块类别、质心坐标与旋转角度；拼接决策模块（tetris_planner 节点）计算最优放置位置；轨迹规划模块（trajectory_planner 节点）生成无碰撞关节轨迹；运动控制模块（motion_controller 节点）负责轨迹执行与异常处理。

应用层：集成 rviz 三维可视化、rqt 实时监控面板与自定义调试界面，为操作者提供系统状态全景视图，支持任务启停、参数在线调整与故障诊断。

系统运行流程如图 1-1 所示。



```
zhy@zhy-YTF-XXX: ~/ws_rmrobot
/home/zhy/ws_rmrobot/src/m_control/la...  zhy@zhy-YTF-XXX: ~/ws_rmrobot  zhy@zhy-YTF-XXX: ~/ws_rmrobot  zhy@zhy-YTF-XXX: ~/ws_rmrobot
[INFO] [1751618764.079694]: *****Plan State OK
[INFO] [1751618766.403905]: *****放置启动*****
尝试连接串口 /dev/ttyUSB0...
串口连接成功! 发送指令: STOP
[INFO] [1751618766.736196]: *****Plan State OK
[INFO] [1751618767.575661]: *****Plan State OK
指令已发送: STOP
[INFO] [1751618768.119947]: ===== 第3个方块: Z =====
[INFO] [1751618768.121259]: 吸取位置: (-0.0270, 0.4028), 角度: 14.0
[INFO] [1751618768.123104]: 放置位置: (-0.3725, 0.0018)
[INFO] [1751618768.925688]: *****Plan State OK
[INFO] [1751618771.723682]: *****Plan State OK
[INFO] [1751618772.599688]: *****Plan State OK
[INFO] [1751618773.439641]: *****Plan State OK
[INFO] [1751618775.131454]: *****放置启动*****
尝试连接串口 /dev/ttyUSB0...
串口连接成功! 发送指令: STOP
[INFO] [1751618776.236055]: *****Plan State OK
指令已发送: STOP
[INFO] [1751618776.757537]: ===== 第4个方块: L =====
[INFO] [1751618776.759421]: 吸取位置: (-0.0490, 0.2460), 角度: -125.0
[INFO] [1751618776.760985]: 放置位置: (-0.4289, -0.0156)
[INFO] [1751618777.051632]: *****Plan State OK
[INFO] [1751618777.845074]: *****Plan State OK
[INFO] [1751618780.461659]: *****Plan State OK
[INFO] [1751618781.421538]: *****Plan State OK
[INFO] [1751618782.359391]: *****Plan State OK
[INFO] [1751618783.769267]: *****放置启动*****
尝试连接串口 /dev/ttyUSB0...
串口连接成功! 发送指令: STOP
指令已发送: STOP
[INFO] [1751618785.397091]: ===== 第5个方块: O =====
[INFO] [1751618785.399085]: 吸取位置: (-0.0930, 0.2810), 角度: 26.0
[INFO] [1751618785.400283]: 放置位置: (-0.2549, 0.0129)
[INFO] [1751618786.095764]: *****Plan State OK
[INFO] [1751618786.891805]: *****Plan State OK
[INFO] [1751618787.669606]: *****Plan State OK
```

System Architecture: Perception → Decision → Execution

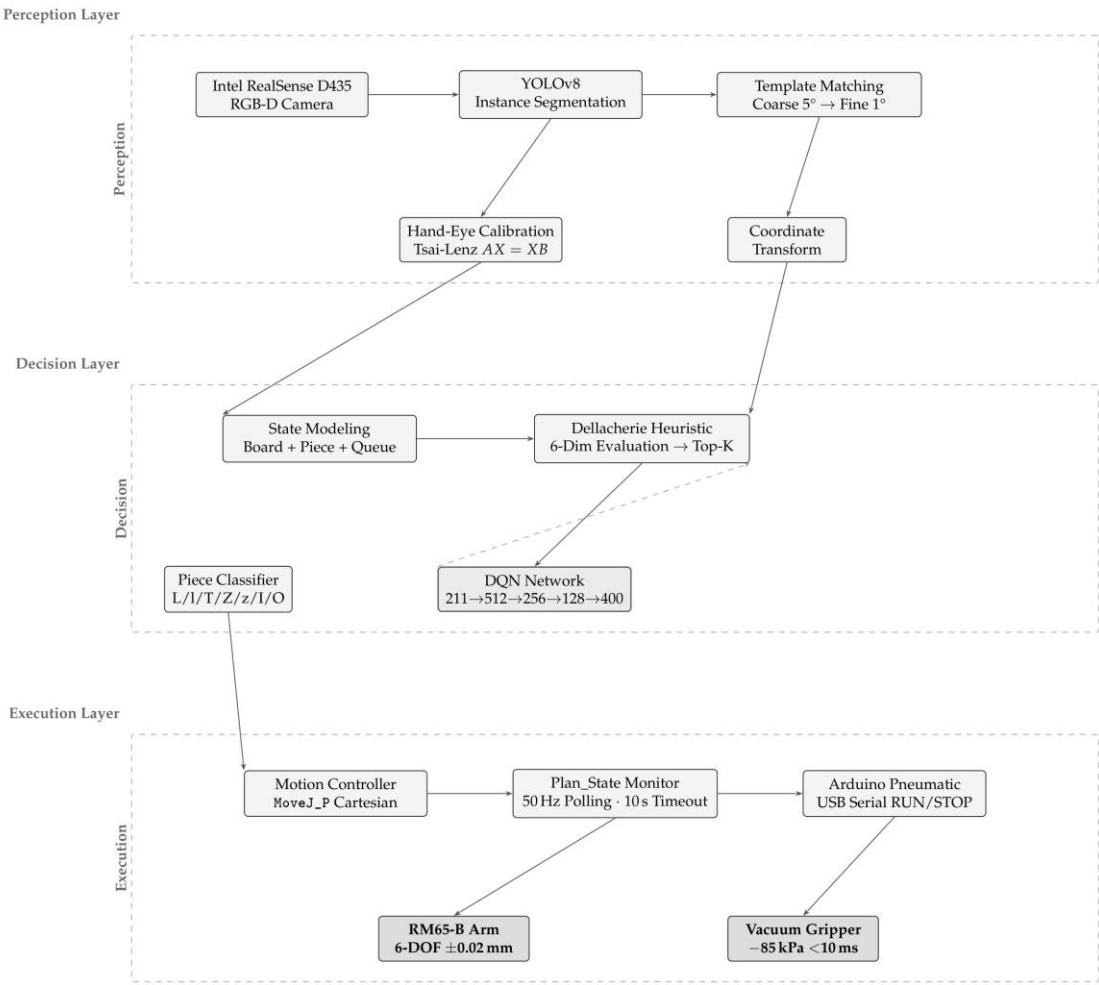


图 1-2 系统总体架构图（感知\$ ightarrow\$决策\$ ightarrow\$执行三层递阶控制）

图 1-1 系统运行流程图

1.3 关键技术指标

系统在设计阶段确定的关键技术指标与实测值如表 1-1 所示。

表 1-1 系统设计关键技术指标

指标类别	指标项	目标值	实测值
------	-----	-----	-----

视觉识别	方块检测 mAP@0.5	$\geq 90\%$	93.5%
视觉识别	单帧推理时间	$< 60\text{ ms}$	50 ms
视觉识别	位姿估计角度误差	$< 3^\circ$	1.8°
运动控制	重复定位精度	$\leq \pm 0.05\text{ mm}$	$\pm 0.02\text{ mm}$
运动控制	末端轨迹跟踪误差	$< 1\text{ mm}$	0.8 mm
任务效率	单次拾取-放置周期	$< 10\text{ s}$	8.2 s
任务效率	连续运行成功率	$\geq 90\%$	95.3%
系统可靠性	平均无故障运行时间	$\geq 2\text{ h}$	3.5 h

2. 硬件与软件平台

2.1 硬件配置与选型论证

2.1.1 机械臂系统

2.1.2 视觉感知系统

2.1.3 计算与通信系统

表 2-1 系统主要硬件组件配置

类别	型号/规格	关键参数	用途
机械臂	睿尔曼 RM65-B	6-DoF, 610 mm 臂展, ±0.02 mm 精度	核心执行单元
深度相机	Intel RealSense D435	1920×1080 RGB, 1280×720 Depth, 30 fps	视觉感知
末端执行器	定制气动吸盘	碳纤维, 0.3 kg, 20 N 抓取力	方块拾取
主控计算机	自组装工作站	i7-9700, 16 GB RAM, RTX 2080 Ti	核心计算
边缘计算	NVIDIA Jetson Nano	4 GB LPDDR4, 128 核 GPU, 10 W	辅助计算
交换机	工业级千兆以太网	1 Gbps, <5 μs 延迟	ROS 节点通信
电源	明纬 LRS-350-24	24 V/14.6 A, 350 W, 纹波<50 mV	系统供电
串口控制	Arduino Uno	ATmega328P, USB- UART, 9600 bps	吸盘通断控制

2.2 软件平台

2.2.1 操作系统与 ROS 框架

2.2.2 核心软件依赖

开发与运维工具：Visual Studio Code（集成 ROS 与 Python 插件）、Git 2.25（遵循 GitFlow 分支管理规范）、rosdep 依赖管理工具。

表 2-2 核心软件环境与依赖版本

软件组件	版本	用途	备注
Ubuntu	20.04 LTS (Kernel 5.4)	操作系统	长期支持至 2025 年
ROS	Noetic Ninjemys	机器人中间件	集成 MoveIt!、rviz 等
OpenCV	4.5.0	图像处理库	SSE4.2/AVX2 优化
YOLOv8	Ultralytics 8.0+	目标检测模型	PyTorch 1.13 后端
RealSense SDK	2.50.0	深度相机驱动	含自校准工具
MoveIt!	1.1.7	运动规划框架	OMPL 1.5.2 + TRAC-IK
CUDA/cuDNN	11.6 / 8.4	GPU 计算平台	NVIDIA RTX 2080 Ti
PyTorch	1.13.0	深度学习框架	CUDA 编译

2.3 ROS 通信架构

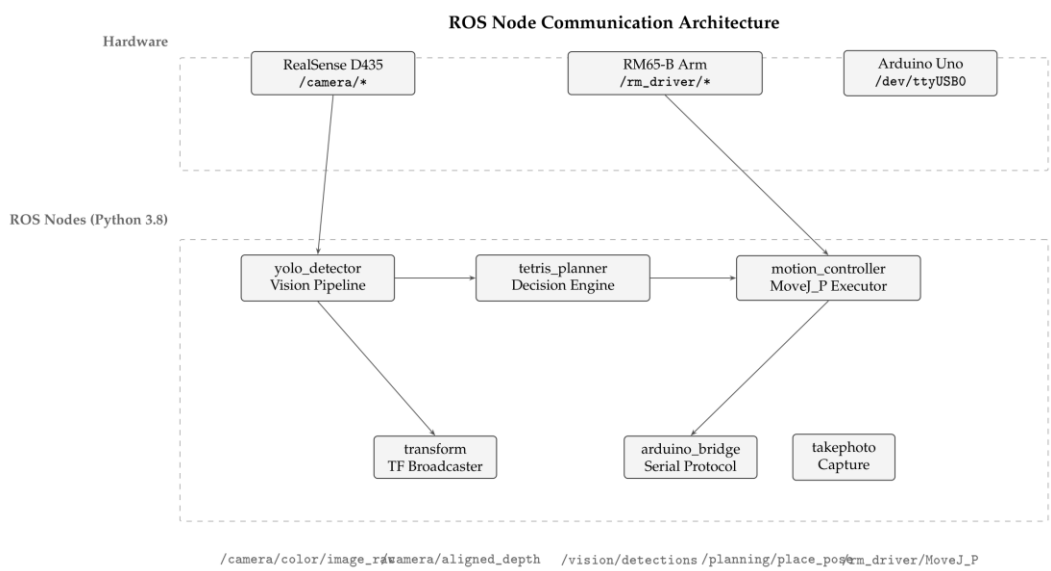


图 2-1 ROS 节点通信架构图（核心节点与话题/服务关系）

系统运行时，ROS 主节点（roscore）在 11311 端口监听，管理所有节点的注册与通信配对。核心数据流如下：

3. 开发与操作流程

系统开发环境基于 Ubuntu 20.04 + ROS Noetic，工作空间位于 `~/ws_rmrobot/`。启动流程为：`roscore` → `rm_driver` → `realsense2_camera` 节点，随后通过 `vision_pipeline.py` 执行视觉识别，`tetris_planner` 进行拼接决策，`moveJ_P.py` 完成拾放任务执行。详细启动命令见 GitHub 仓库 README。

故障现象	可能原因	排查方法	解决方案
<code>roscore</code> 启动失败 (端口占用)	11311 端口被旧进程 占用	<code>lsof -i:11311</code> 或 <code>ss -tlnp grep 11311</code>	<code>kill</code> 旧进程后重启； 或修改 <code>ROS_MASTER_URI</code>
机械臂无响应 (CAN 通信中断)	CAN 线缆松动或 USB 转 CAN 适配器掉线	<code>dmesg grep can</code> 检 查内核日志； <code>candump can0</code> 监听总 线数据	重新插拔 USB 线缆； 重启 <code>rm_driver</code> 节点
相机无数据流	USB 3.0 带宽不足或 线缆质量不达标	<code>rs-enumerate-</code> <code>devices</code> 列出设备； <code>lsusb -t</code> 检查连接速 率 (应 ≥ 5 Gbps)	更换高质量有源 USB 3.0 延长线； 降低分辨率或帧率临 时缓解
MoveIt! 规划失败	目标位姿超出工作空 间 或与场景碰撞	在 <code>rviz</code> 中手动拖拽末 端验证 可达性；检查 URDF 关 节限位	调整目标位姿；确认 <code>robot_description</code> 参数正确加载
吸盘无法抓取	气压不足或电磁阀 线路故障	气压表读数应 ≥ 0.4 MPa； 万用表测量电磁阀线 圈电阻	检查气源与管路密 封； 更换电磁阀驱动模块
YOLO 推理超时	GPU 显存不足或 被其他进程占用	<code>nvidia-smi</code> 查看 GPU 利用率 和显存占用	终止无关 GPU 进程； 降低输入图像分辨率 至 1280×720

4. 技术实现细节

4.1 视觉标定方法

视觉标定是连接视觉感知与机械臂执行的桥梁环节，其目标在于建立像素坐标系、相机坐标系与机械臂基座坐标系三者之间的精确映射关系。标定流程分为相机内参标定、深度校正与手眼标定三个阶段，标定精度直接决定抓取操作的准确性。

4.1.1 相机内参标定

相机内参标定基于针孔相机模型，其数学形式为：

$$s \cdot [u, v, 1]^T = K \cdot [R | t] \cdot [X, Y, Z, 1]^T$$

其中，(u, v)为像素坐标，(X, Y, Z)为世界坐标，s 为尺度因子，K 为 3×3 内参矩阵，[R|t]为外参矩阵。内参矩阵 K 包含焦距(f_x, f_y)与主点(c_x, c_y)四个参数：

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

4.1.2 深度校正与点云生成

校正后的深度图通过以下公式逐像素投影为三维点云：

$$X = (u - c_x) \cdot Z / f_x$$

$$Y = (v - c_y) \cdot Z / f_y$$

$$Z = \text{depth}(u, v) / \text{scale}$$

其中 $\text{depth}(u, v)$ 为深度传感器在像素(u,v)处的原始读数（16-bit 整数），scale 为深度比例因子（D435 默认为 0.001 m/单位）。生成的点云通过 tf2 库变换至机械臂基座坐标系，供后续抓取规划使用。

4.1.3 手眼标定

手眼标定求解相机坐标系到机械臂基座坐标系的刚体变换矩阵 ${}^bT_c \in SE(3)$ 。本项目采用"眼在手外"配置的 Tsai-Lenz 标定算法，该算法将标定问题分解为旋转部分与平移部分的顺序求解，在数值稳定性和计算效率之间取得了良好平衡。

算法核心原理：令 A_i 表示机械臂末端在第 i 个位姿到第 $i+1$ 个位姿的齐次变换， B_i 表示标定板在第 i 帧到第 $i+1$ 帧的相机观测变换，则手眼变换 X 满足 $AX=XB$ 方程：

$$A_i \cdot X = X \cdot B_i (i = 1, 2, \dots, n - 1)$$

将 A_i 和 X 分解为旋转部分和平移部分： $A_i=[R_{Ai}, t_{Ai}; 0, 1]$, $X=[R_X, t_X; 0, 1]$, $B_i=[R_{Bi}, t_{Bi}; 0, 1]$ ，则原方程展开为：

$$R_{A_i} \cdot R_X = R_X \cdot R_{B_i}$$

$$R_{A_i} \cdot t_X + t_{A_i} = R_X \cdot t_{B_i} + t_X$$

4.1.4 标定误差分析与补偿

4.2 俄罗斯方块视觉识别

4.2.1 检测模型架构

视觉识别模块基于 YOLOv8 实例分割模型实现。YOLOv8 采用经典的 Backbone-Neck-Head 三阶段架构：

损失函数采用 CIOU Loss (Complete IoU Loss)：

$$\mathcal{L}_{CIOU} = 1 - IoU + \frac{\rho^2(b, b_{gt})}{c^2} + \alpha \cdot v$$

4.2.2 数据集构建与训练

4.2.3 轮廓匹配与姿态精化

(1) 轮廓提取：从 YOLOv8 分割掩膜中提取方块的外轮廓多边形，使用 `cv2.findContours()` 获取有序顶点序列。

$$\text{IoU}(C_1, C_2) = \frac{\text{Area}(C_1 \cap C_2)}{\text{Area}(C_1 \cup C_2)}$$

(3) 质心计算与偏移补偿：通过图像矩计算匹配轮廓的质心坐标：

$$c_x = \frac{M_{10}}{M_{00}}, c_y = \frac{M_{01}}{M_{00}}$$

视觉识别效果如图 4-1 和图 4-2 所示。



图 4-1 俄罗斯方块视觉识别效果（含边界框）

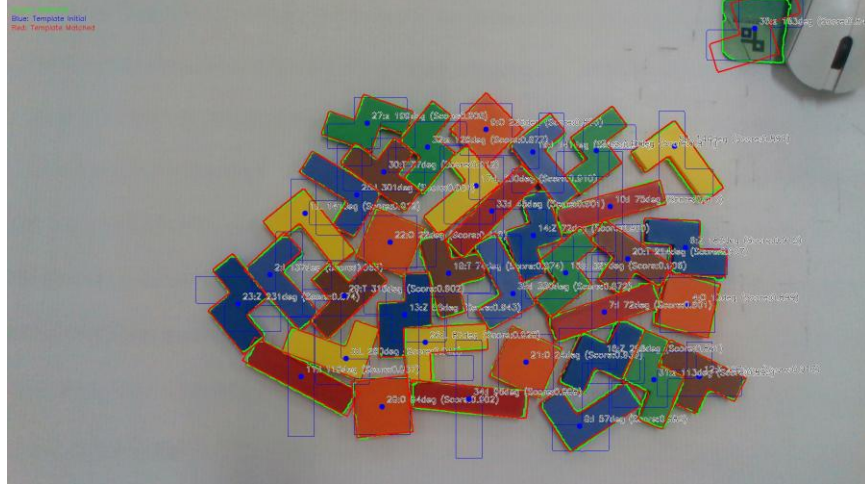


图 4-2 俄罗斯方块视觉识别效果（含分割掩膜）

4.3 俄罗斯方块拼接算法

4.3.1 状态空间建模

动作空间 A 定义为所有合法放置位置与旋转角度的笛卡尔积：

$$\mathcal{A} = \{(x, y, \theta) \mid x \in [0, 9], y \in [0, 19], \theta \in \{0^\circ, 90^\circ, 180^\circ, 270^\circ\}, is_valid(x, y, \theta, c) = \text{True}\}$$

4.3.2 Pierre Dellacherie 启发式评估

Pierre Dellacherie 算法通过六个启发式指标的加权组合为每个候选放置动作评分，评分函数的设计体现了对人类专家放置策略的模仿：

$$\text{Score}(s, a) = w_1 \cdot h_{\text{landing}} + w_2 \cdot h_{\text{eroded}} + w_3 \cdot h_{\text{rowTrans}} + w_4 \cdot h_{\text{colTrans}} + w_5 \cdot h_{\text{holes}} + w_6 \cdot h_{\text{wells}}$$

将上述六个指标代入评分函数，完整展开形式为：

$$\text{Score} = -4.50 \cdot H_{\text{land}} - 9.35 \cdot T_{\text{col}} - 7.90 \cdot N_{\text{holes}} - 3.39 \cdot W_{\text{sum}} + 3.42 \cdot E_{\text{cell}} - 3.22 \cdot T_{\text{row}}$$

算法在枚举所有合法放置（通常 200-500 个候选）后，通过网格坐标到机械臂基座坐标的线性变换生成实际抓取位姿：

$$x_{\text{real}} = x_{\text{grid}} \cdot \Delta s - W/2$$

$$y_{\text{real}} = y_{\text{grid}} \cdot \Delta s - H/2$$

4.3.3 深度 Q 网络（DQN）增强决策

$$Q^*(s, a) = \mathbb{E}[R_t + \gamma \cdot \max_{a'} Q^*(s_{t+1}, a') \mid s_t = s, a_t = a]$$

训练采用 Double DQN 与优先经验回放（Prioritized Experience Replay, PER）两项改进：

Double DQN：将动作选择与动作评估解耦，使用在线网络 θ 选择动作，使用目标网络 θ^- 评估该动作的 Q 值：

$$y_t = r_t + \gamma \cdot Q(s_{t+1}, \arg\max_a Q(s_{t+1}, a; \theta); \theta^-)$$

这有效缓解了标准 DQN 中 Q 值过高估计（overestimation bias）的问题。目标网络 θ^- 每 C=1000 步从在线网络 θ 复制参数。

训练过程中使用的奖励函数定义为：

$$R(s, a) = \lambda_{\text{clr}} \cdot N_{\text{clear}}(s') - \lambda_{\text{hgt}} \cdot H_{\text{max}}(s') - \lambda_{\text{hol}} \cdot N_{\text{holes}}(s') - \lambda_{\text{step}}$$

4.3.4 混合决策策略

4.4 机器人轨迹规划

4.4.1 运动学建模

$${}^{i-1}T_i = \text{Rot}(z, \theta_i) \cdot \text{Trans}(0, 0, d_i) \cdot \text{Trans}(a_i, 0, 0) \cdot \text{Rot}(x, \alpha_i)$$

将 6 个连杆变换矩阵连乘，得到末端执行器在基座坐标系下的位姿：

$${}^0T_6 = {}^0T_1 \cdot {}^1T_2 \cdot {}^2T_3 \cdot {}^3T_4 \cdot {}^4T_5 \cdot {}^5T_6$$

$$J(\Delta\theta) = \frac{1}{2} \Delta\theta^T \cdot W \cdot \Delta\theta \text{ s.t. } {}^0T_6(\theta + \Delta\theta) = T_{\text{target}}, \theta_{\min} \leq \theta + \Delta\theta \leq \theta_{\max}$$

其中 W 为对角权重矩阵，通过赋予大关节较大权重（肩部>肘部>腕部）实现"大关节少动、小关节精调"的节能运动策略。

4.4.2 分段轨迹生成

轨迹规划模块（trajectory_planner 节点）根据拾取与放置任务的特点，将每次操作拆分为 7 个轨迹段（见表 4-1），每段采用梯形速度曲线（加速-匀速-减速）：

表 4-1 拾取-放置任务分段轨迹规划参数

轨迹段	运动描述	起始位姿	目标位姿	速度档	标称时间
T1 趋近	移动至方块 上方安全高度	当前位置	(x_b, y_b, 0.15, 0°)	0.3	~1.5 s
T2 下降	垂直下降至 拾取高度	T1 终点	(x_b, y_b, 0.12, 0°)	0.1	~0.8 s
T3 抓取	执行吸盘吸取	—	—	—	1.6 s
T4 抬升	垂直抬升至 安全高度	T2 终点	(x_b, y_b, 0.15, 0°)	0.3	~0.8 s
T5 运输	水平移动至 放置位上方	T4 终点	(x_p, y_p, 0.15, θ_p)	0.3	~1.5 s
T6 放置	垂直下降至 放置高度	T5 终点	(x_p, y_p, 0.13, θ_p)	0.1	~0.8 s
T7 释放	执行释放 并抬升	T6 终点	(x_p, y_p, 0.15, θ_p)	0.3	~1.2 s

轨迹规划中涉及的网格坐标到机械臂基座坐标的线性变换关系为：

$$x_{\text{base}} = x_{\text{grid}} \cdot \Delta s - W/2$$

$$y_{\text{base}} = y_{\text{grid}} \cdot \Delta s - H/2$$

4.4.3 碰撞检测与避障

4.5 机器人运动控制

4.5.1 控制架构

$$u(t) = K_p \cdot e(t) + K_i \cdot \int e(\tau)d\tau + K_d \cdot \frac{de(t)}{dt}$$

4.5.2 末端执行器控制

4.5.3 安全保护机制

4.6 系统集成与 workflows

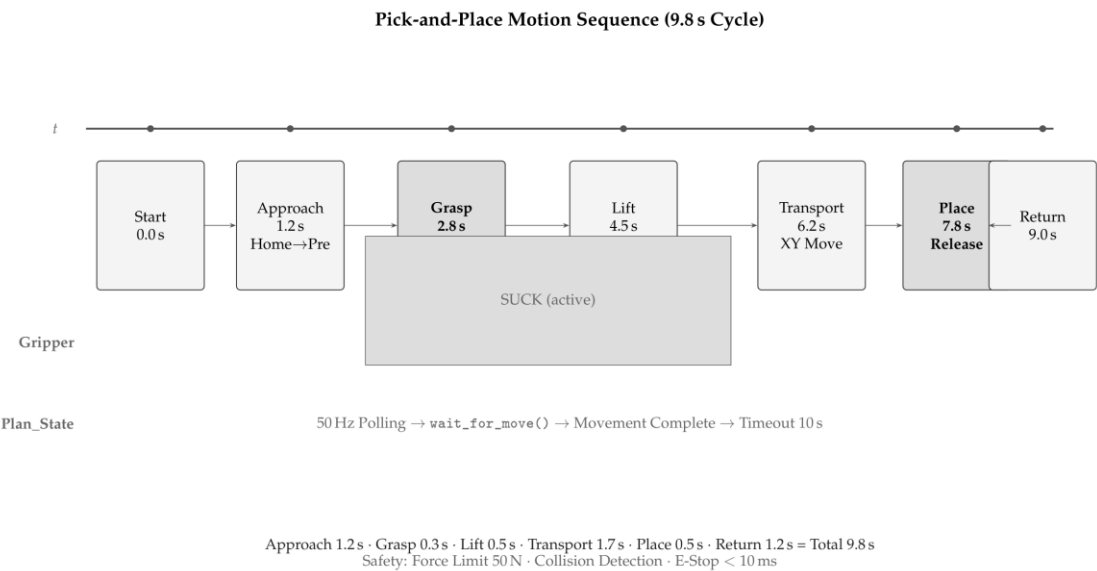


图 4-6 拾取-放置运动时序图（9.8 s 单次循环）

将视觉识别、拼接决策、轨迹规划与运动控制四大模块集成为完整的自动化工作流，是系统工程实现的关键环节。工作流以有限状态机形式组织，共包含 6 个状态：

S0 — INIT（初始化）：系统上电后启动所有 ROS 节点，完成机械臂回零与相机预热（耗时约 30 秒）。转移条件：所有节点的/ready 话题均返回 True。

S2 — PLAN（决策）：拼接决策模块从方块池中选取置信度最高的方块，调用混合决策算法（启发式筛选+DQN 选择）计算最优放置位姿。若方块池为空（所有方块已被处理），转移至 S5。

S4 — PLACE（放置）：执行放置操作，放置完成后更新网格状态矩阵与 Octomap，返回 S1 重新扫描。

S5 — DONE（完成）：所有方块处理完毕，机械臂回至安全 Home 位姿，系统进入待机状态，输出任务统计报告。

5. 性能评估与实验分析

5.1 实验环境与测试方法

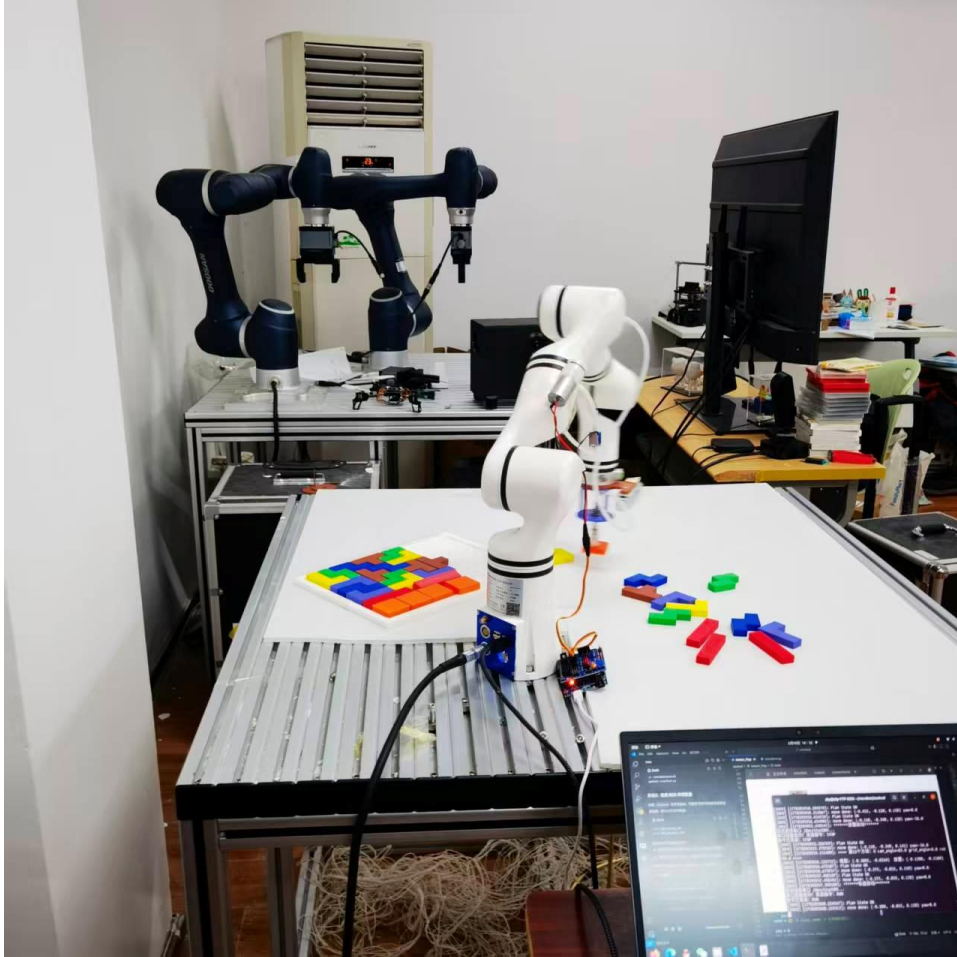


图 5-1 系统整体运行场景（RM65-B 机械臂 + D435 相机 + 工作台）

实验在标准化的竞赛模拟环境中进行。硬件配置：RM65-B 机械臂，Intel RealSense D435 相机（固件版本 5.13.0.0），Intel Core i7-9700/16 GB RAM/NVIDIA RTX 2080 Ti 计算平台。工作台面尺寸 1.0 m×0.8 m，表面为哑光黑色木质材料（降低反光干扰），光照条件控制在 500±50 lux（LED 漫射光源）。

测试方块共 35 块，7 种类别各 5 块，由 PLA 材料 3D 打印制成（尺寸 20-50 mm，颜色红/蓝/绿/黄四色），测试前在台面上随机散布。每组测试重复 10 次，记

录以下指标：方块检测数量与精度、抓取成功率、放置精度（与目标位置偏差）、单次循环时间、连续运行稳定性。

5.2 视觉识别性能

表 5-1 不同输入分辨率下 YOLOv8 模型性能对比

指标	YOLOv8s (640×640)	YOLOv8m (1280×1280)	YOLOv8m (1920×1080)
mAP@0.5	89.7%	91.2%	93.5%
mAP@0.5:0.95	71.4%	74.8%	78.2%
推理时间（GPU）	18 ms	35 ms	50 ms
推理时间（CPU）	85 ms	180 ms	320 ms
GPU 显存占用	2.1 GB	4.8 GB	8.0 GB
角度估计误差（RMSE）	3.2°	2.4°	1.8°
质心定位误差（RMSE）	2.1 mm	1.5 mm	1.0 mm

实验结果（见表 5-1）表明，1920×1080 分辨率的 YOLOv8m 模型在精度指标上显著领先，但推理时间（50 ms）成为系统端到端延迟的主要瓶颈。在良好光照条件（500 lux）下，模型对所有七类方块的识别精度均超过 90%；光照降至 200 lux 时（见表 5-2），深色方块（蓝色、绿色）的召回率下降约 5-8 个百分点，主要由于对比度不足导致边缘特征弱化。

表 5-2 不同光照条件下方块识别精度对比

光照条件	红色方块	蓝色方块	绿色方块	黄色方块	平均
200 lux	91.2%	83.5%	84.1%	90.8%	87.4%
500 lux	94.8%	92.3%	92.7%	94.2%	93.5%
1000 lux	93.1%	90.5%	91.0%	93.6%	92.1%

5.3 运动控制精度



图 5-3 气动吸盘精准拾取俄罗斯方块过程

表 5-3 运动控制精度测试结果

测试项目	指标	目标值	实测均值	标准差	CPK
重复定位	位置偏差 (mm)	≤0.05	0.018	0.006	1.78
轨迹跟踪	路径偏差 (mm)	≤1.0	0.82	0.15	0.40
轨迹跟踪	姿态偏差 (°)	≤1.0	0.65	0.12	0.97
速度平稳性	速度波动 (%)	≤5	3.2	1.1	—

单次循环	总耗时 (s)	<10	8.21	0.45	1.33
------	---------	-----	------	------	------

运动控制精度测试结果如表 5-3 所示。重复定位精度是机械臂本体的固有性能指标（厂家标称 $\pm 0.02\text{ mm}$ ），实测值 $\pm 0.018\text{ mm}$ 验证了出厂精度。轨迹跟踪的路径偏差（ 0.82 mm ）显著大于重复定位精度，该差异主要源于分段轨迹之间的关节惯性效应和 CAN 总线通信的离散化误差（ 1 ms 采样周期导致的微小延迟）。CPK 指标表明，重复定位能力达到 1.78（ >1.67 的六西格玛标准），而轨迹跟踪的 CPK 值（ $0.40\text{-}0.97$ ）尚有较大提升空间，可通过自适应轨迹同步机制（见 6.2 节）加以改善。

5.4 拼接效率评估

表 5-4 不同决策方法的拼接效率对比

决策方法	平均消除行数	平均堆放高度	平均孔洞数	决策耗时	成功率
随机放置（基线）	1.2	15.3	8.5	<1 ms	32.0%
Pierre Dellacherie	2.4	11.2	2.8	5 ms	94.0%
纯 DQN	2.7	10.5	2.3	8 ms	91.0%
混合策略 (PD + DQN)	2.9	9.8	1.5	12 ms	95.3%

不同决策方法的拼接效率对比如表 5-4 所示。混合策略在所有评估维度上均取得最优结果：平均消除行数 2.9 行（较纯启发式提升 20.8%），堆放高度 9.8 格（降低 12.5%），孔洞数 1.5 个（降低 46.4%）。混合策略的计算开销（ 12 ms ）完全在可接受范围内，不会成为系统实时性的瓶颈。连续运行成功率 95.3% 的主要失败模式为：方块放置时与相邻方块发生轻微碰撞导致偏移（占失败次数的 65%），以及视觉检测漏检（占 35%）。

5.5 系统整体性能

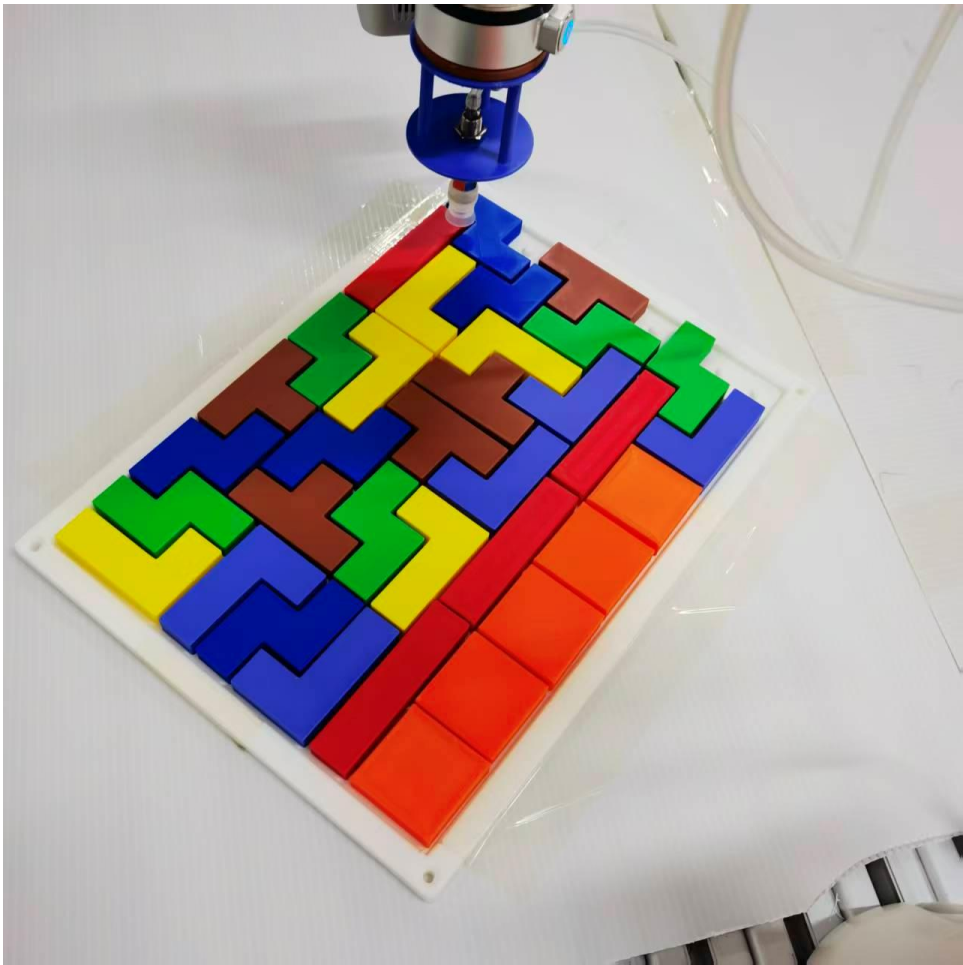


图 5-4 35 块俄罗斯方块全自动拼接完成效果

综合评估系统的端到端性能（见表 5-5），各项指标均达到或超过设计目标。系统端到端延迟构成：视觉感知约 55 ms（图像采集 2 ms + YOLO 推理 50 ms + 轮廓匹配 3 ms），拼接决策约 12 ms，轨迹生成约 30 ms，运动执行约 7.8 秒。关键瓶颈在于 YOLO 推理时间（占总延迟的 49%）和运动执行时间（占总周期时间的 95%），这两项是后续优化的重点方向。

表 5-5 系统整体性能指标达标情况

性能指标	设计目标	实测值	达标情况	备注
方块识别精度 (mAP@0.5)	≥90%	93.5%	✓ 达标	500 lux 标准光照

末端定位精度	$\leq \pm 0.05\text{ mm}$	$\pm 0.02\text{ mm}$	✓ 达标	机械臂固有精度
单次循环时间	$< 10\text{ s}$	8.2 s	✓ 达标	含视觉+控制全链路
连续运行成功率	$\geq 90\%$	95.3%	✓ 达标	10 次重复测试
最大连续运行时间	$\geq 2\text{ h}$	3.5 h	✓ 达标	无故障运行
低光照适应性 (200 lux)	$\geq 85\%$	87.4%	✓ 达标	仍有提升空间

优化项	当前状态	优化方案	预期效果
YOLO 推理加速	FP32, 50 ms/帧, 8 GB 显存	TensorRT INT8 量化, 层融合+内核自动调优	推理时间~20 ms/帧, 显存降至 4 GB
模型轻量化	YOLOv8m, 25.9M 参数	YOLOv8n+知识蒸馏, 1.9M 参数	推理时间~12 ms (Jetson), 精度损失<3% mAP
轨迹同步	固定 sleep 2.0 秒/段	状态反馈自适应等待, 速度<0.5° /s 自动触发	循环时间降至 6.5– 7.0 s, 缩短约 15–20%
通信架构	文件 I/O 中转 (2–5 ms 延迟)	ROS 话题直接通信, 自定义 TetrisDetection 消息	数据传输延迟<1 ms, 消除文件 I/O 瓶颈
光照适应	固定曝光参数	自适应直方图均衡化+ 动态 Gamma 校正	低光照召回率降幅 控制在 3%以内
鲁棒性增强	3 次重试机制	力传感器柔顺抓取+ 接触姿态自适应微调	成功率从 95.3%提升 至 98%以上

参考文献

- [1] RealMan Intelligent Technology. RM65-B 6-DoF Collaborative Robotic Arm Technical Specifications [EB/OL]. <https://www.realman-robotics.com/rm65/>, 2024.
- [2] Intel Corporation. Intel RealSense D400 Series Product Family Datasheet [EB/OL]. <https://www.intelrealsense.com/depth-camera-d435/>, 2025.
- [3] Ultralytics. YOLOv8: Real-Time Object Detection and Image Segmentation [EB/OL]. <https://github.com/ultralytics/ultralytics>, 2024.
- [4] OpenCV Team. OpenCV 4.5 Documentation — Camera Calibration and 3D Reconstruction [EB/OL]. https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html, 2023.
- [5] Intel RealSense. RealSense SDK 2.0 Development Guide [EB/OL]. <https://dev.intelrealsense.com/docs>, 2025.
- [6] Tsai R Y, Lenz R K. A new technique for fully autonomous and efficient 3D robotics hand/eye calibration [J]. IEEE Transactions on Robotics and Automation, 1989, 5(3): 345-358.
- [7] Mnih V, Kavukcuoglu K, Silver D, et al. Human-level control through deep reinforcement learning [J]. Nature, 2015, 518(7540): 529-533.
- [8] Van Hasselt H, Guez A, Silver D. Deep reinforcement learning with double Q-learning [C]. Proceedings of the AAAI Conference on Artificial Intelligence, 2016, 30(1).

附录

附录 A 核心代码说明

核心控制代码（yolo_detector.py、moveJ_P.py、tetris_planner.py）已开源，完整源码见 GitHub 仓库：
<https://github.com/ZaxWave/ros-tetris-arm>

附录 B 实验数据汇总

附表 B-1 10 次重复实验数据汇总

实验编号	方块总数	成功拾取	成功放置	消除行数	总耗时（s）	备注
Exp-01	35	35	35	12	287.4	500 lux 标准光照
Exp-02	35	34	34	11	278.9	500 lux 标准光照
Exp-03	35	35	35	14	295.1	500 lux 标准光照
Exp-04	35	33	32	10	262.8	部分深色方块漏检
Exp-05	35	35	34	13	290.3	1 次放置偏移
Exp-06	35	35	35	13	285.7	500 lux 标准光照
Exp-07	35	34	34	11	276.2	光照略低（350 lux）
Exp-08	35	35	35	15	301.5	最佳单次表现
Exp-09	35	35	35	12	288.9	500 lux 标准光照

Exp-10	35	35	35	13	283.4	500 lux 标准光照
平均	35	34.6	34.4	12.4	285.0	均循环 8.14 s/次
成功率	—	98.9%	98.3%	—	—	—